Волков Михаил Викторович, ИУ5-35Б

Отчёт по Рубежному Контролю №1 по курсу ПИК ЯП

Условия рубежного контроля №1 по курсу ПиК ЯП

Рубежный контроль представляет собой разработку программы на языке Python, которая выполняет следующие действия:

1) Необходимо создать два класса данных в соответствии с Вашим вариантом предметной области, которые связаны отношениями один-ко-многим и многие-ко-многим.

Пример классов данных для предметной области Сотрудник-Отдел:

1. Класс «Сотрудник», содержащий поля:
   - ID записи о сотруднике;
   - Фамилия сотрудника;
   - Зарплата (количественный признак);
   - ID записи об отделе. (для реализации связи один-ко-многим)
2. Класс «Отдел», содержащий поля:
   - ID записи об отделе;
   - Наименование отдела.
3. (Для реализации связи многие-ко-многим) Класс «Сотрудники отдела», содержащий поля:
   - ID записи о сотруднике;
   - ID записи об отделе.

2) Необходимо создать списки объектов классов, содержащих тестовые данные (3-5 записей), таким образом, чтобы первичные и вторичные ключи соответствующих записей были связаны по идентификаторам.

3) Необходимо разработать запросы в соответствии с Вашим вариантом. Запросы сформулированы в терминах классов «Сотрудник» и «Отдел», которые используются в примере. Вам нужно перенести эти требования в Ваш вариант предметной области. При разработке запросов необходимо по возможности использовать функциональные возможности языка Python (list/dict comprehensions, функции высших порядков).

Для реализации запроса №2 введите в класс, находящийся на стороне связи «много», произвольный количественный признак, например, «зарплата сотрудника».

Результатом рубежного контроля является документ в формате PDF, который содержит текст программы и результаты ее выполнения.

Вариант запросов: Г

Вариант Г.

1. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список всех отделов, у которых название начинается с буквы «А», и список работающих в них сотрудников.
2. «Отдел» и «Сотрудник» связаны соотношением один-ко-многим. Выведите список отделов с максимальной зарплатой сотрудников в каждом отделе, отсортированный по максимальной зарплате.
3. «Отдел» и «Сотрудник» связаны соотношением многие-ко-многим. Выведите список всех связанных сотрудников и отделов, отсортированный по отделам, сортировка по сотрудникам произвольная.

Вариант предметной области: 17

17	Дирижер	Оркестр

Код программы

```python
from dataclasses import dataclass
from typing import List


# 1. ОПИСАНИЕ КЛАССОВ ДАННЫХ

@dataclass
class Orchestra:
    """Класс «Оркестр» (Сторона «Один»)"""
    id: int
    name: str


@dataclass
class Conductor:
    """Класс «Дирижёр» (Сторона «Много»)"""
    id: int
    fullname: str
    experience_years: int  # Количественный признак (аналог зарплаты)
    orchestra_id: int  # Внешний ключ (ID оркестра)


@dataclass
class OrchestraConductorLink:
    """Класс для связи Многие-ко-многим"""
    conductor_id: int
    orchestra_id: int
```

```python
# 2. СОЗДАНИЕ ТЕСТОВЫХ ДАННЫХ

def create_test_data():
    # Создаем оркестры
    orchestras = [
        Orchestra(1, "Академический симфонический"),
        Orchestra(2, "Большой театр"),
        Orchestra(3, "Австрийский филармонический"),
        Orchestra(4, "Джазовый бэнд")
    ]

    # Создаем дирижеров (один-ко-многим: у каждого дирижера есть
основной оркестр)
    conductors = [
        Conductor(1, "Иванов И.И.", 15, 1),
        Conductor(2, "Петров П.П.", 8, 2),
        Conductor(3, "Сидоров С.С.", 20, 1),
        Conductor(4, "Смит Дж.", 12, 3),
        Conductor(5, "Вагнер Р.", 25, 3)
    ]

    # Создаем связи многие-ко-многим (кто где когда-либо дирижировал)
    links = [
        OrchestraConductorLink(1, 1),  # Иванов -> Академический
        OrchestraConductorLink(3, 1),  # Сидоров -> Академический
        OrchestraConductorLink(2, 2),  # Петров -> Большой театр
        OrchestraConductorLink(4, 3),  # Смит -> Австрийский
        OrchestraConductorLink(5, 3),  # Вагнер -> Австрийский
        OrchestraConductorLink(2, 4)  # Петров -> Джазовый бэнд
    ]

    return orchestras, conductors, links


# 3. РЕАЛИЗАЦИЯ ЗАПРОСОВ

def query_1(orchestras: List[Orchestra], conductors: List[Conductor]):
    """
    Запрос 1. Вывести список всех оркестров, название которых
начинается с «А»,
    и список работающих в них дирижеров (связь один-ко-многим).
    """
    print("\n--- Запрос 1: Оркестры на 'A' и их дирижеры ---")

    # 1. Находим оркестры на 'A' через list comprehension
    target_orchestras = [o for o in orchestras if
o.name.startswith("A")]

    # 2. Для каждого найденного оркестра собираем дирижеров
    for orch in target_orchestras:
```

```
        print(f"Оркестр: {orch.name}")

        # Функциональный подход: filter по orchestra_id
        orch_conductors = list(filter(lambda c: c.orchestra_id ==
orch.id, conductors))

        if not orch_conductors:
            print("  (Нет дирижеров)")
        else:
            for c in orch_conductors:
                print(f"  - {c.fullname} (стаж: {c.experience_years}
лет)")


def query_2(orchestras: List[Orchestra], conductors: List[Conductor]):
    """
    Запрос 2. Вывести список оркестров с максимальным стажем дирижера
в каждом оркестре,
    отсортированный по максимальному стажу.
    """
    print("\n--- Запрос 2: Оркестры по макс. стажу дирижера ---")

    result = []

    for orch in orchestras:
        # Собираем стаж всех дирижеров этого оркестра
        exp_list = [c.experience_years for c in conductors if
c.orchestra_id == orch.id]

        # Если в оркестре есть дирижеры, берем максимальный стаж
        if exp_list:
            max_exp = max(exp_list)
            result.append((orch.name, max_exp))
        else:
            result.append((orch.name, 0))

    # Сортируем по стажу (по убыванию). key=lambda x: x[1] указывает
сортировать по 2-му элементу кортежа
    sorted_result = sorted(result, key=lambda x: x[1], reverse=True)

    for name, max_exp in sorted_result:
        print(f"{name}: макс. стаж {max_exp} лет")


def query_3(orchestras: List[Orchestra], conductors: List[Conductor],
links: List[OrchestraConductorLink]):
    """
    Запрос 3. Вывести список всех связанных дирижеров и оркестров
(многие-ко-многим),
    отсортированный по названиям оркестров.
```

```
    """
    print("\n--- Запрос 3: Связи многие-ко-многим (по оркестрам) ---")

    # Создаем быстрые словари для поиска по ID за O(1)
    orch_dict = {o.id: o.name for o in orchestras}
    cond_dict = {c.id: c.fullname for c in conductors}

    # Формируем список строк для вывода
    output_data = [
        (orch_dict[link.orchestra_id], cond_dict[link.conductor_id])
        for link in links
    ]

    # Сортируем по названию оркестра (первый элемент кортежа)
    sorted_links = sorted(output_data, key=lambda x: x[0])

    for orch_name, cond_name in sorted_links:
        print(f"Оркестр: {orch_name:35} | Дирижер: {cond_name}")


# 4. ЗАПУСК
if __name__ == "__main__":
    orch_data, cond_data, link_data = create_test_data()

    print("Данные загружены:")
    print(f"Оркестров: {len(orch_data)}")
    print(f"Дирижеров: {len(cond_data)}")

    query_1(orch_data, cond_data)
    query_2(orch_data, cond_data)
    query_3(orch_data, cond_data, link_data)
```

```
Данные загружены:
Оркестров: 4
Дирижеров: 5

--- Запрос 1: Оркестры на 'А' и их дирижеры ---
Оркестр: Академический симфонический
  - Иванов И.И. (стаж: 15 лет)
  - Сидоров С.С. (стаж: 20 лет)
Оркестр: Австрийский филармонический
  - Смит Дж. (стаж: 12 лет)
  - Вагнер Р. (стаж: 25 лет)

--- Запрос 2: Оркестры по макс. стажу дирижера ---
Австрийский филармонический: макс. стаж 25 лет
Академический симфонический: макс. стаж 20 лет
Большой театр: макс. стаж 8 лет
Джазовый бэнд: макс. стаж 0 лет

--- Запрос 3: Связи многие-ко-многим (по оркестрам) ---
```

```
Оркестр: Австрийский филармонический          | Дирижер: Смит Дж.
Оркестр: Австрийский филармонический          | Дирижер: Вагнер Р.
Оркестр: Академический симфонический          | Дирижер: Иванов И.И.
Оркестр: Академический симфонический          | Дирижер: Сидоров С.С.
Оркестр: Большой театр                        | Дирижер: Петров П.П.
Оркестр: Джазовый бэнд                        | Дирижер: Петров П.П.
```

Результат работы программы

